Lumina: Personal Laptop AI Assistant

Comprehensive Technical Plan & Architecture Document

Project Owner: Your Name

June 8, 2026

Contents

1 Introduction

1.1 Project Vision

Lumina is a deeply integrated, locally-aware AI assistant that lives inside a personal laptop. It is not merely a chatbot or a thin wrapper around LLM APIs; it is an **autonomous agent runtime and orchestration platform** designed from the ground up for full system control, persistent memory, context-rich coding assistance, and extensible tool use. The primary goal is to learn systems engineering, backend architecture, distributed coordination, memory retrieval, low-level sandboxing, and AI orchestration — while building something genuinely useful.

1.2 Core Design Philosophy

- **Build, don't bolt**: Every subsystem is designed and implemented from scratch where it teaches the most (e.g., sandboxed shell execution, memory engine, event bus). Avoids over-reliance on pre-built frameworks.
- **Language-appropriate**: Use Python for AI glue and rapid prototyping, Go for high-concurrency backend services, and Rust for low-level systems components where safety and performance are paramount.
- **Secure by default**: No direct agent-to-shell access. Every tool call passes through a permission layer, validation, and sandboxing.
- **Scalable memory**: Context management is not just vector search; it uses hierarchical, multi-modal retrieval that respects workspace boundaries, temporal dynamics, and semantic compression.
- **Event-driven backbone**: All internal changes (shell output, file modification, agent lifecycle) are emitted as events, enabling decoupled, observable, and recoverable workflows.

2 High-Level Architecture

Lumina is organized as a layered system of seven core layers, each composed of independent, well-defined services.

1. **Agent Runtime Layer** – Orchestrates planning, tool routing, memory access, context assembly, and multi-agent scheduling.
2. **Tooling / Capability Layer** – Provides isolated, auditable tools for shell, file system, git, email, browser, etc.
3. **Memory & Context Layer** – Multi-modal memory: episodic, semantic, workspace-specific, execution memory, and long-term summaries.
4. **Sandbox Execution Layer** – Low-level process isolation (Linux namespaces, seccomp, cgroups) managed through a Rust executor.

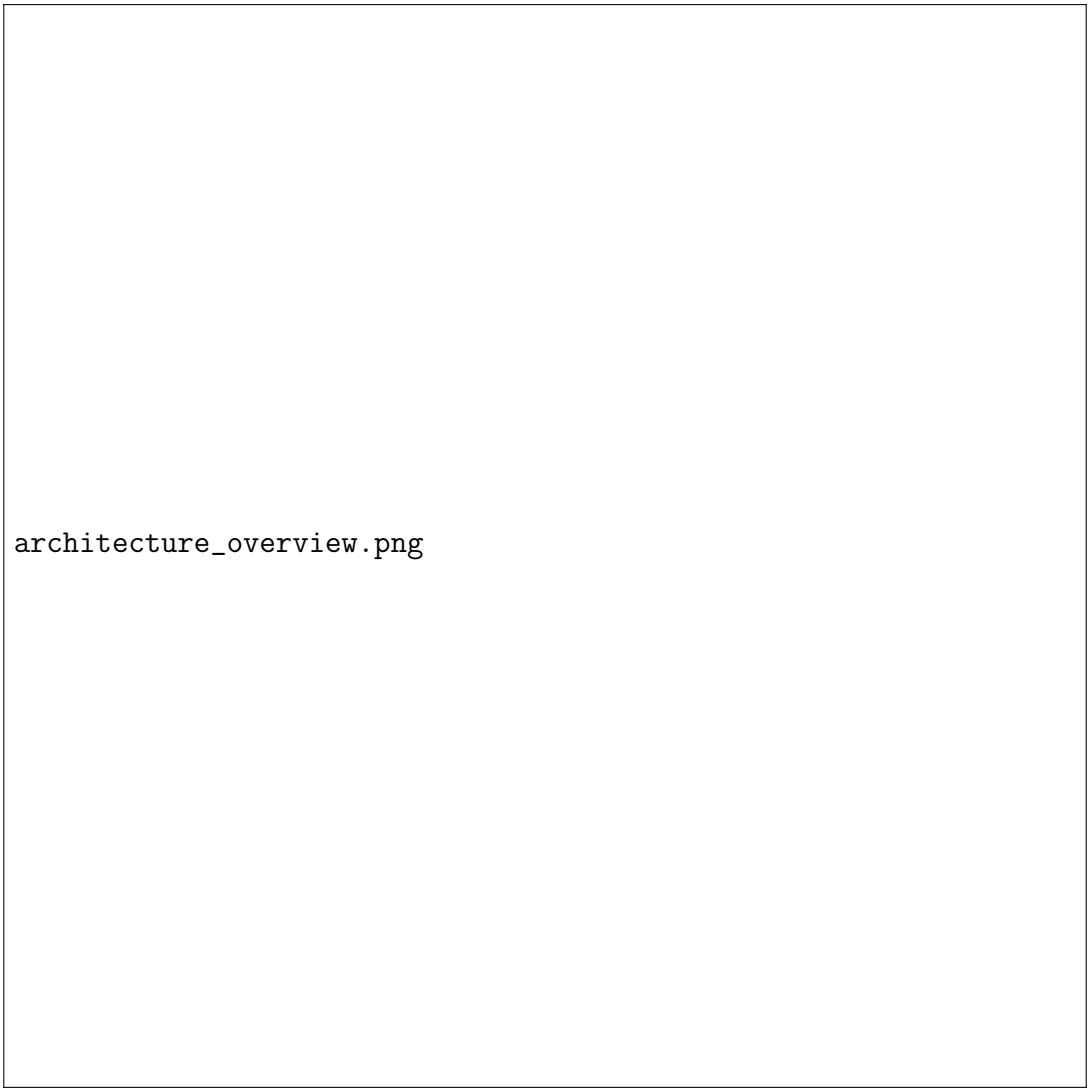

Figure 1: Lumina high-level architecture layers.

5. **AI Orchestration Layer** – Model-agnostic routing, planning, tool calling, and fallback logic.

6. **Coding Agent Infrastructure** – Codebase indexing, AST parsing, symbol graphs, diff generation, and build pipelines.

7. **Event-Driven Backend** – Asynchronous event bus connecting all modules, backed by PostgreSQL and message queues.

3 Detailed Subsystem Design

3.1 Agent Runtime Layer

The runtime is the brain of Lumina. It manages the lifecycle of agents and their interactions with tools and memory.

3.1.1 Core Components

- **Planner**: Decomposes user intents into task graphs. Supports hierarchical planning with replanning on failure.
- **Tool Router**: Maps tool calls to appropriate backend implementations, enforcing permissions and versioning.
- **Memory Engine**: Retrieves relevant context from the multi-modal memory system (see Section **??**).
- **Context Builder**: Assembles the final prompt by merging user query, tool definitions, retrieved memory, and workspace state.
- **Sandbox Executor**: Forwards validated tool calls (e.g., shell commands) to the secure execution layer.
- **Event Bus Client**: Publishes events (agent.started, tool.called, error.occurred) for observability and recovery.
- **Permission Layer**: Checks capabilities and user-defined policies before any tool execution.
- **Multi-Agent Scheduler**: Coordinates concurrent sub-agents, handling resource allocation and task dependencies.

3.2 Tooling / Capability Layer

Every tool is a first-class entity with a strict interface.

3.2.1 Tool Manifest (Example)

```
{
  "name": "file_read",
  "description": "Read a file within allowed workspaces.",
  "parameters": {
    "path": {"type": "string", "description": "Absolute path to file"}
  },
  "permissions": ["filesystem.read"],
  "sandbox": "none",  // runs in Rust sandboxed executor
  "backend": "python" // or "rust" for low-level ops
}
```

Listing 1: Tool definition schema

3.2.2 Implemented Tools (Phase 1)

- **Shell**: Execute commands in a containerised shell; capture stdout/stderr, exit codes.
- **File System**: Read, write, append files; list directories; watch for changes (via inotify).
- **Git**: Commit, diff, push/pull, branch management.
- **Email**: Send/receive via SMTP/IMAP; can be expanded to WhatsApp, Signal, etc.

- **Browser**: Puppeteer-like control for web research and form filling.
- **Local Apps**: Launch, focus, and send keystrokes to any desktop application.

Critical safety rule: *Never directly expose an unrestricted shell.* All commands are templated, validated against a whitelist/blacklist, and executed inside a container/-namespace with limited capabilities.

3.3 Memory & Context Layer

This subsystem is the intellectual differentiator of Lumina. It enables the agent to maintain coherent, long-term context across sessions and tasks without bloating the prompt.

3.3.1 Memory Categories

- **Episodic Memory**: Logs of past interactions – shell outputs, agent actions, user corrections. Stored in PostgreSQL (structured timeline) and MongoDB (raw JSON documents).
- **Semantic Memory**: Factual knowledge about the user, projects, and concepts. Stored as embeddings + graph relationships.
- **Workspace Memory**: Per-project context: git diffs, build commands, API specs, dependency trees. Indexed by path and symbol.
- **Execution Memory**: A temporal trace of all tool calls, their inputs/outputs, and performance metrics. Used for healing and optimisation.
- **Long-Term Summary Memory**: Periodic compression of episodic memory into high-level summaries, stored with recursive LLM-based condensation.

3.3.2 Hierarchical Context Retrieval Pipeline

Instead of naive top-k vector search, Lumina uses a multi-stage pipeline:

1. **Intent Classification**: Identify whether query is about code, past execution, system state, etc.
2. **Memory Type Selection**: Choose which memory sources to query.
3. **Workspace Detection**: Determine which project/repo the query concerns (via active window, file paths, user hint).
4. **Temporal Filtering**: Prioritise recent events or specific time windows.
5. **Semantic Search + Hybrid Retrieval**: Combine vector similarity, keyword BM25, and graph traversal.
6. **Context Compression**: Summarise or trim redundant information using a lightweight local model.
7. **Reranking**: Cross-encoder model to order candidate chunks by relevance.
8. **Prompt Assembly**: Merge retrieved context with system instructions, tool definitions, and current query.

3.3.3 Storage Backend

- **PostgreSQL**: Primary DB for structured metadata, user sessions, agent states, tool logs.
- **MongoDB**: Unstructured/flexible storage for full chat history, raw tool outputs, and evolving memory documents.
- **Qdrant (or Chroma)**: Vector database for semantic embeddings. Initially local, with option to scale.
- **Redis**: Caching and pub/sub for real-time events.

3.4 Sandbox Execution Layer

The shell utility must be *air-gapped* from the host.

3.4.1 Rust-Based Executor

A dedicated Rust service manages:

- Linux namespaces (mount, PID, network, user) for full container-like isolation.
- seccomp-bpf filters to whitelist/blacklist system calls.
- cgroups v2 to limit CPU, memory, and I/O.
- PTY allocation for interactive commands (pseudo-terminal).
- Streaming output back to the agent via gRPC or WebSockets.

3.4.2 Execution Flow

1. Agent sends a shell tool call (command string, timeout, working directory).
2. Permission layer validates that the command is allowed.
3. Command is dispatched to the Rust executor, which creates a new namespace and pty session.
4. Stdout/stderr are streamed in real time; exit code is captured.
5. The executor returns the result and logs the event.

3.5 AI Orchestration Layer

Lumina must not be tied to a single LLM provider.

3.5.1 Model Abstraction

A unified interface (‘LLMProvider‘) that routes requests to any backend:

- OpenAI, Anthropic, Google Gemini, Groq
- Local models via Ollama, LM Studio, vLLM, llama.cpp
- Self-hosted endpoints

```
def generate(messages: list, tools: list = None, stream: bool = False,
             model: str = "gpt-4o", provider: str = "openai") ->
    Response:
    ...
```

Listing 2: Unified LLM call

3.5.2 Agent Planning & Tool Use

Agents follow a ReAct/Plan-and-Execute loop. The planner can decompose complex tasks (e.g., “analyse this repo and suggest optimisations”) into sub-tasks, each dispatched to specialised sub-agents (code-reader, shell-runner, memory-searcher). Tool calls are parsed, validated, and executed; results are fed back into the context.

3.6 Coding Agent Infrastructure

A major goal is an AI-powered software engineering companion that deeply understands codebases.

3.6.1 Workspace Indexer

- **AST Parsing** with Tree-sitter (Rust, Python, Go, TypeScript, etc.) to extract functions, classes, imports.
- **Symbol Graph**: Build a call graph and data-flow graph for cross-file reasoning.
- **Dependency Analysis**: Track library versions, vulnerable packages, and build configurations.
- **Embedding Generation**: Create vector representations of functions, files, and whole modules using a code-specific embedding model.
- **Semantic Search**: Hybrid search (symbol name + vector similarity) to find relevant code snippets.

3.6.2 Coding Agent Workflows

- **Diff Generation**: Propose changes as unified diffs; agent reviews and explains them.
- **Build & Test**: Execute ‘make‘, ‘cargo build‘, ‘pytest‘ inside sandbox, interpret results.
- **Debugging**: Correlate error logs with code context and recent git changes.
- **Refactoring**: Use AST transformations (via tools like ‘ast-grep‘) for safe, structured changes.

3.7 Event-Driven Backend

Instead of a monolithic API, Lumina uses an internal event bus.

3.7.1 Event Categories

- `shell.started`, `shell.completed`, `shell.failed`
- `file.created`, `file.modified`, `file.deleted`
- `git.commit`, `git.pushed`
- `agent.spawned`, `agent.task.completed`, `agent.error`
- `memory.created`, `context.updated`
- `ui.input`, `ui.output`

Events are produced by all services and consumed by:

- **Logging & Observability** (Elasticsearch/Kibana or simple file logger)
- **Self-Healing** (see Section **??**)
- **Memory Engine** (populate episodic/execution memory)
- **Notification Service** (push to user's phone, email)

The backbone can be implemented with Redis Streams, NATS, or RabbitMQ, with PostgreSQL used for durable event storage.

3.8 Healing Mode

Self-repair is an ambitious feature, but must be constrained for safety.

3.8.1 Constrained Healing Protocol

1. **Detect Failure**: Agent monitors its own error logs and user feedback.
2. **Analyse**: Read its own source code (with read-only permission) and recent execution traces.
3. **Propose Patch**: Generate a git diff for the fix.
4. **Test**: Run the test suite in a sandbox; reject if tests fail.
5. **Request Approval**: Display the diff and test results to the user; do not apply automatically.
6. **Apply**: If approved, apply the patch and restart the affected service.

No direct self-modification of running binaries is ever allowed. The healing loop is always supervised by the user.

Layer	Language	Rationale
API / Orchestration	Python (FastAPI)	Rich AI ecosystem, rapid proto typing, async support.
High-Concurrency Services	Go	Efficient goroutines, excelle for event streaming, scheduler backend workers.
Systems / Sandboxing	Rust	Memory safety, zero-cost abstra tions, perfect for low-level proces isolation and PTY management
Frontend (optional)	TypeScript + React/Tauri	If a desktop GUI is ever buil Tauri uses Rust for native bind ings.
Databases	PostgreSQL, MongoDB, Qdrant, Redis	Structured state, flexib documents, vector searc caching/pub-sub.
Message Broker	Redis Streams or NATS	Lightweight, high-throughput i ternal event bus.

Table 1: Recommended language choices per subsystem.

4 Technology Stack

The stack is deliberately multi-language to maximise learning.

5 Development Roadmap

5.1 Phase 1 – Core Runtime (Weeks 1–4)

Goal: A minimal but functional agent that can chat and run safe shell commands.

- Set up FastAPI project with basic CRUD for conversations.
- Implement ‘LLMProvider‘ abstraction (OpenAI + local Ollama).
- Build shell execution tool with Rust sandbox (basic namespace isolation).
- Create a simple file reader/writer tool with permission checks.
- Add token-based authentication and session management.
- Integrate PostgreSQL for user/session storage.

5.2 Phase 2 – Memory System (Weeks 5–8)

Goal: Agents remember past interactions and project context.

- Set up MongoDB for chat history and unstructured tool logs.
- Implement episodic memory storage and retrieval.

- Integrate Qdrant and compute embeddings for semantic memory.
- Build context retrieval pipeline (intent → workspace → hybrid search → compression).
- Add workspace-specific memory (per-project git logs, diffs).

5.3 Phase 3 – Coding Agent (Weeks 9–14)

Goal: An agent that understands your code and can make safe modifications.

- Implement Tree-sitter-based code indexer for Python, Rust, Go.
- Create symbol graphs and semantic search over code.
- Build diff generation tool and pull-request style review.
- Integrate git operations (commit, push, branch).
- Add build/test execution in sandbox.

5.4 Phase 4 – Multi-Agent & Event Backend (Weeks 15–20)

Goal: True orchestration with concurrent agents and an event-driven backbone.

- Design internal event bus with Redis/NATS.
- Implement planner that splits complex tasks into sub-agents.
- Add multi-agent scheduler and resource manager.
- Migrate tool logs and state transitions to events.
- Build basic observability dashboard (logs, metrics).

5.5 Phase 5 – Local OS Control (Weeks 21–26)

Goal: Full laptop integration.

- Add email (IMAP/SMTP) tool, WhatsApp (via web automation), calendar.
- File system watcher and reactive workflows (e.g., "when a PDF is added to Downloads, summarise it").
- Desktop automation (mouse/keyboard control via Python's 'pyautogui' or Rust 'enigo').
- Permission profiles (read-only, sandboxed, unrestricted – user defined).

5.6 Phase 6 – Advanced Features (Ongoing)

- Healing mode with supervised patching.
- Long-horizon planning (multi-day tasks, background monitoring).
- Distributed execution (offload heavy tasks to a home server or cloud).
- Voice interface: integrate on-device STT (whisper.cpp) and TTS (Piper or Coqui).
- Memory evolution: adaptive compression and knowledge graph merging.

6 Learning Outcomes

By building Lumina incrementally, you will gain deep experience in:

- **Backend engineering**: REST/gRPC APIs, database design, authentication, message queues.
- **Distributed systems**: Event-driven architectures, service decoupling, eventual consistency.
- **Systems programming**: Linux namespaces, seccomp, file descriptors, PTY, process management (Rust).
- **AI/ML infrastructure**: Embedding models, vector databases, RAG pipelines, model routing.
- **Orchestration**: Multi-agent coordination, planning, tool-use frameworks.
- **Developer tools**: ASTs, symbol graphs, static analysis, continuous integration.
- **Observability**: Structured logging, metrics, tracing, and self-monitoring.

7 Conclusion

Lumina is more than a personal assistant — it is a platform for exploring the frontier of autonomous systems, memory-centric AI, and systems-level programming. By adhering to a disciplined, layer-by-layer approach, the project will remain tractable while growing into a genuinely powerful tool. The focus on safety, multi-language learning, and from-scratch implementation ensures that every module contributes to a deeper understanding of how modern AI and backend systems truly work.

This document is a living blueprint; it should be revised as design decisions are made and new technologies evaluated.